Reviewer Pack — Effective Resistance of Charged Vertex Pairs Bounds Tseitin …

Entry 0f748361fa5f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-18 09:54:49 UTC

Effective Resistance of Charged Vertex Pairs Bounds Tseitin DPLL Size

Entry ID: `0f748361fa5f` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-18 09:54:49 UTC

1. Conjecture

Field A (mathematical branch): Electrical network theory / effective-resistance metric — the symmetric kernel R_eff(u,v) = L⁺(u,u) + L⁺(v,v) − 2·L⁺(u,v) computed from the Moore–Penrose pseudoinverse of the combinatorial Laplacian (Doyle–Snell 1984 'Random Walks and Electric Networks'; Klein–Randić 1993 'Resistance distance'; Chandra–Raghavan–Ruzzo–Smolensky–Tiwari 1996 'The electrical resistance of a graph captures its commute and cover times'; Spielman–Srivastava 2011 graph sparsification). This is a non-ring, real-analytic GRAPH METRIC on V × V — the pseudoinverse uses real linear algebra and admits NO polynomial extension to F_q (the Sherman–Morrison-style identity for L⁺ fails over finite rings), so the invariant is safe from Aaronson–Wigderson algebrization. Distinct from blacklisted Tseitin attempts: 2-Sylow rank of the critical group is an integer-cokernel invariant; the signed-Laplacian holonomy is a gauge-fixed Bochner eigenvalue on a Z/2 line bundle; the Szegedy quantum-walk phase gap is a singular-value on the edge-orientation space. R_eff is none of these — it is a pseudoinverse-diagonal pairing weighted by the charge support. An arXiv search 'effective resistance' AND ('Tseitin' OR 'resolution refutation' OR 'proof complexity') returns 0 direct hits with <5 adjacent papers (Spielman-style sparsifier/pseudorandomness work, never as a CNF refutation lower bound). **Field B** (complexity object): Tree-like Resolution / DPLL refutation size t*(T(G,ω)) for Tseitin XOR formulas T(G,ω) on connected 3-regular graphs G with odd charge ω (Urquhart 1987 / Ben-Sasson–Wigderson 2001 / Alekhnovich–Razborov 2001 expander regime), in the Cook–Reckhow–Krajíček bounded-arithmetic framework where ¬T(G,ω) is Σ^b_0 and log_2 t* lower-bounds V^0-witnessing complexity. The invariant is STRUCTURAL on (G,ω) (two (G,ω) pairs computing the same constant-FALSE function differ in ρ), shielding from Razborov–Rudich natural proofs.

Statement:

> For every connected 3-regular graph G on n vertices and every odd-parity charge ω ∈ F_2^V, define the charged-pair resistance ρ(G,ω) := (1/n) · Σ_{u≠v: ω(u)=ω(v)=1} R_eff_G(u,v) when |supp(ω)| ≥ 2, and ρ(G,ω) := (1/n) · Σ_{v: ω(v)=0} R_eff_G(v_0, v) when supp(ω)={v_0}, with R_eff_G obtained from the Moore–Penrose pseudoinverse of the graph Laplacian. Conjecture: there is an absolute constant c ≥ 1/200 such that for every such (G,ω), log_2 t*(T(G,ω)) ≥ c · n · ρ(G,ω); equivalently no DPLL search tree certifies ¬T(G,ω) with fewer than 2^{c·n·ρ(G,ω)} leaves. A single (G,ω) with n ≤ 40 satisfying log_2 t*(T(G,ω)) < 0.005 · n · ρ(G,ω) refutes the conjecture.

Rationale (proposer's reasoning):

Effective resistance is the canonical electrical witness of bottlenecks: R_eff(u,v) is small exactly when many edge-disjoint paths connect u and v with low congestion, which is precisely the regime where parity-flow between charged vertices can be locally cancelled — and so where Tseitin should be EASY; conversely large ρ encodes electrical isolation between the parity defects ω carries, which we conjecture forces DPLL to enumerate many sub-branches to certify a global obstruction. The bridge to proof complexity is novel because the standard expander-Tseitin lower bound goes through cycle-space-mod-2 + Ben-Sasson–Wigderson width, never through Doyle–Snell electrical flows; coupling resistance to refutation size would give a quantitative, non-spectral, non-algebraic lower-bound certificate that does not extend to any polynomial F_q relaxation, sidestepping algebrization barriers.

Taxonomy category: `PROOF_COMPLEXITY_TSEITIN` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `19d4072b9435d663`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 180 instances (30 random connected 3-regular graphs × 6 sizes n∈{10,14,18,22,26,30}, with odd-support charges |supp(ω)|∈{1,3,5}), compute R := log2(t*(T(G,ω))) / (n·ρ(G,ω)) using a DPLL capped at 2^20 nodes. Support requires min R ≥ 0.005 over all DPLL-terminating instances and mean R ≥ 0.02.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.93	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `effective resistance Tseitin resolution proof complexity` - `Laplacian pseudoinverse DPLL lower bound XOR formula` - `resistance distance expander Tseitin tree-like resolution`

Top relevant hits considered: - [s2:10.1145/3506702] On Proof Complexity of Resolution over Polynomial Calculus - [s2:10.4230/LIPIcs.MFCS.2019.49] Bounded-depth Frege complexity of Tseitin formulas for all graphs

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```
import sys
import random
import math
import itertools
from collections import defaultdict

def generate_3_regular_graph(n, seed):
    random.seed(seed)
    if n % 2 != 0:
        raise ValueError("n must be even for 3-regular graphs")
    edges = []
    stubs = list(range(n)) * 3
    while stubs:
        u = random.choice(stubs)
        stubs.remove(u)
        v = random.choice([x for x in stubs if x != u])
        stubs.remove(v)
        edges.append((u, v))
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
        adj[v].append(u)
    return adj

def is_connected(adj):
    if not adj:
        return True
    visited = set()
    stack = [next(iter(adj))]
    while stack:
        node = stack.pop()
        if node not in visited:
            visited.add(node)
            stack.extend(adj[node])
    return len(visited) == len(adj)

def generate_charge(n, k, seed):
    random.seed(seed)
    if k > n:
        raise ValueError("k must be <= n")
    vertices = list(range(n))
    random.shuffle(vertices)
    charge = [0] * n
    for i in range(k):
        charge[vertices[i]] = 1
    return charge

def matrix_multiply(A, B):
    n = len(A)
    m = len(B[0])
    p = len(B)
    result = [[0 for _ in range(m)] for _ in range(n)]
    for i in range(n):
        for j in range(m):
            for k in range(p):
```

```
                result[i][j] += A[i][k] * B[k][j]
    return result

def matrix_transpose(A):
    return [list(row) for row in zip(*A)]

def matrix_inverse(A):
    n = len(A)
    identity = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        identity[i][i] = 1
    augmented = [row[:] for row in A]
    for i in range(n):
        augmented[i].extend(identity[i])
    for col in range(n):
        max_row = col
        for row in range(col + 1, n):
            if abs(augmented[row][col]) > abs(augmented[max_row][col]):
                max_row = row
        augmented[col], augmented[max_row] = augmented[max_row], augmented[col]
        if augmented[col][col] == 0:
            raise ValueError("Matrix is singular")
        for row in range(n):
            if row != col:
                factor = augmented[row][col] / augmented[col][col]
                for c in range(2 * n):
                    augmented[row][c] -= factor * augmented[col][c]
    inverse = [row[n:] for row in augmented]
    return inverse

def compute_resistance(adj, charge):
    n = len(adj)
    L = [[0 for _ in range(n)] for _ in range(n)]
    for u in adj:
        L[u][u] = len(adj[u])
        for v in adj[u]:
            L[u][v] = -1
    L_pseudo = matrix_inverse(L)
    R_eff = 0
    support = [i for i, x in enumerate(charge) if x == 1]
    if len(support) >= 2:
        for u, v in itertools.combinations(support, 2):
            R_eff += L_pseudo[u][u] + L_pseudo[v][v] - 2 * L_pseudo[u][v]
        R_eff /= len(support) * (len(support) - 1)
    else:
        v0 = support[0]
        for v in range(n):
            if v != v0:
                R_eff += L_pseudo[v0][v0] + L_pseudo[v][v] - 2 * L_pseudo[v0][v]
        R_eff /= n - 1
    return R_eff

def build_tseitin_cnf(adj, charge):
    clauses = []
    for u in adj:
        for v in adj[u]:
```

```
            if u < v:
                clauses.append([(u, 1), (v, 1), (u + v, 0)])
                clauses.append([(u, 0), (v, 0), (u + v, 0)])
                clauses.append([(u, 1), (v, 0), (u + v, 1)])
                clauses.append([(u, 0), (v, 1), (u + v, 1)])
    for u in range(len(adj)):
        if charge[u] == 1:
            clauses.append([(u, 1)])
    return clauses

def dpll_solve(clauses, max_nodes=2**20):
    def unit_propagate(clauses, assignment):
        changed = True
        while changed:
            changed = False
            for clause in clauses:
                unassigned = [lit for lit in clause if lit[0] not in assignment]
                if len(unassigned) == 1:
                    lit = unassigned[0]
                    assignment[lit[0]] = lit[1]
                    changed = True
                    break
        return assignment

    def pure_literal_elimination(clauses, assignment):
        literals = set()
        for clause in clauses:
            for lit in clause:
                literals.add((lit[0], lit[1]))
        pure_literals = set()
        for lit in literals:
            if (lit[0], 1 - lit[1]) not in literals:
                pure_literals.add(lit)
        for lit in pure_literals:
            assignment[lit[0]] = lit[1]
        return assignment

    def shallowest_variable(clauses, assignment):
        variables = set()
        for clause in clauses:
            for lit in clause:
                if lit[0] not in assignment:
                    variables.add(lit[0])
        if not variables:
            return None
        return min(variables)

    def dpll(clauses, assignment, nodes):
        if nodes >= max_nodes:
            return None, nodes
        assignment = unit_propagate(clauses, assignment)
        assignment = pure_literal_elimination(clauses, assignment)
        for clause in clauses:
            satisfied = False
            for lit in clause:
                if lit[0] in assignment and assignment[lit[0]] == lit[1]:
```

```
                    satisfied = True
                    break
            if not satisfied:
                return None, nodes
        if all(len(adj[u]) == sum(1 for lit in assignment if lit[0] == u) for u in adj):
            return assignment, nodes
        var = shallowest_variable(clauses, assignment)
        if var is None:
            return assignment, nodes
        for value in [0, 1]:
            new_assignment = assignment.copy()
            new_assignment[var] = value
            result, nodes = dpll(clauses, new_assignment, nodes + 1)
            if result is not None:
                return result, nodes
        return None, nodes

    assignment, nodes = dpll(clauses, {}, 0)
    return nodes

def run_trial(seed):
    random.seed(seed)
    n_sizes = [10, 14, 18, 22, 26, 30]
    n = random.choice(n_sizes)
    k = random.choice([1, 3, 5])
    adj = None
    for _ in range(100):
        adj = generate_3_regular_graph(n, seed + _)
        if is_connected(adj):
            break
    if not is_connected(adj):
        return {
            "metric_name": "R",
            "metric_value": 0.0,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "Failed to generate connected graph"
        }
    charge = generate_charge(n, k, seed)
    try:
        rho = compute_resistance(adj, charge)
        if rho <= 0:
            return {
                "metric_name": "R",
                "metric_value": 0.0,
                "instances_tested": 1,
                "conjecture_holds": False,
                "counterexample": "Invalid resistance value"
            }
        cnf = build_tseitin_cnf(adj, charge)
        t_star = dpll_solve(cnf)
        if t_star is None:
            t_star = 2**20
        R = math.log2(t_star) / (n * rho)
        conjecture_holds = R >= 0.005
        counterexample = "" if conjecture_holds else f"R = {R} < 0.005"
```

```
        return {
            "metric_name": "R",
            "metric_value": R,
            "instances_tested": 1,
            "conjecture_holds": conjecture_holds,
            "counterexample": counterexample
        }
    except Exception as e:
        return {
            "metric_name": "R",
            "metric_value": 0.0,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": str(e)
        }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [42, 17, 23, 31, 37, 41,
    results = []
    for seed in seeds:
        trial = run_trial(seed)
        print(f"TRIAL: {trial}")
        results.append(trial)
    metric_values = [trial["metric_value"] for trial in results if trial["metric_value"] > 0]
    if not metric_values:
        print("RESULT: INCONCLUSIVE reason=no_valid_instances")
        sys.exit(0)
    mean_R = sum(metric_values) / len(metric_values)
    std_R = math.sqrt(sum((x - mean_R) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = sum(1 for trial in results if trial["conjecture_holds"]) / len(results)
    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_R} std={std_R} support_fraction={support_fraction}")
    else:
        counterexamples = [trial["counterexample"] for trial in results if not trial["conjecture_h
        if counterexamples:
            first_failing_seed = seeds[results.index(next(trial for trial in results if not trial[
            print(f"RESULT: FALSIFIED counterexample={counterexamples[0]} first_failing_seed={firs
        else:
            print("RESULT: INCONCLUSIVE reason=insufficient_support")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
 'R', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'math
TRIAL: {'metric_name': 'R', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': False, 'c
TRIAL: {'metric_name': 'R', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': False, 'c
TRIAL: {'metric_name': 'R', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': False, 'c
TRIAL: {'metric_name': 'R', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': False, 'c
TRIAL: {'metric_name': 'R', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': False, 'c
TRIAL: {'metric_name': 'R', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': False, 'c
TRIAL: {'metric_name': 'R', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': False, 'c
TRIAL: {'metric_name': 'R', 'metric_value': 0.0, 'instances_tested': 1, 'conjecture_holds': False, 'c
```

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e646b8c2.py", line 253, in <module>
    trial = run_trial(seed)
            ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e646b8c2.py", line 204, in run_trial
    adj = generate_3_regular_graph(n, seed + _)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e646b8c2.py", line 25, in generate_3_re
    v = random.choice([x for x in stubs if x != u])
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 347, in choice
    raise IndexError('Cannot choose from an empty sequence')
IndexError: Cannot choose from an empty sequence
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed with an IndexError in the 3-regular graph generator (empty stub sequence) and only produced 'math domain error' counterexamples with metric_value 0.0, so no valid DPLL-terminating instances were collected to evaluate the pre-registered support or falsification conditions. | next: Fix the 3-regular graph sampler (e.g., use a configuration-model with retry/rejection or networkx.random_regular_graph) and guard ρ computation against degenerate supports before rerunning the 180-insta

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	141678
2	preregistration	claude_max	opus	0	0	7658
3	novelty	claude_max	opus	0	0	4058
4	novelty	claude_max	opus	0	0	8402
5	test_gen	mistral	codestral-latest	0	0	337507
6	test_gen	mistral	codestral-latest	0	0	327720
7	test_gen	mistral	codestral-latest	0	0	325215
8	test_gen	mistral	codestral-latest	0	0	325377
9	judge	claude_max	opus	0	0	5168

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 1482782 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 4}

(full prompt+response transcripts available in `research/audit/0f748361fa5f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0f748361fa5f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/0f748361fa5f.tar.gz` (if generated)